Лабораторная работа №4

Семёнов Александр Дмитриевич

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **9**

4.1 Установка репозиториев. 9
4.2 Общепринятые коммиты. 10
4.3 Создание репозитория git. 11
4.4 Конфигурация git flow. 12
4.5 Работа с репозиторием git. 15

5 Выводы **19**

Список литературы **20**

Список иллюстраций

4.1 Установка из коллекции репозиториев 9
4.2 Установка из коллекции репозиториев 9
4.3 Установка node.js 9
4.4 Установка pnpm 10
4.5 Выполнение команды 10
4.6 Выполнение pnpm add -g commitizen 10
4.7 Выполнение команды pnpm add -g standard-changelog 10
4.8 Созданный репозиторий 11
4.9 Выполнение git commit -m “first commit и git push -u origin master 11
4.10 Выполнение pnpm init и заполнение пакетов 12
4.11 Созданный репозиторий 12
4.12 Коммит 12
4.13 Отправка ни гит 12
4.14 Инициализация 13
4.15 Проверка ветки 13
4.16 Загрузка 13
4.17 установка вышестоящей ветки 13
4.18 Создание релиза 14
4.19 Создание журнала 14
4.20 Добавление журнала в индекс 14
4.21 Залив релизной ветки 14
4.22 Отправка данных 14
4.23 Отправка данных 15
4.24 Создание релиза с помощью утилитов 15
4.25 Создание новой ветки 15
4.26 Продолжение работы 15
4.27 Объединение веток 16
4.28 Создание релиза 16
4.29 Обновление номера версии 16
4.30 Создание журнала 16
4.31 Добавление журнала 17
4.32 Залив ветки 17
4.33 Отправка 17
4.34 Отправка 17
4.35 Создание релиза 18

Список таблиц

1 Цель работы

Получение навыков продвинутой работы с **git** и **релизами**.

2 Задание

Выполнить работу для тестового репозитория и преобразовать рабочий репозиторий в репозиторий с **git-flow** и **conventional commits**.

3 Теоретическое введение

Системы контроля версий (Version Control System, VCS) используются для организации совместной работы коллектива над общим проектом. Как правило, основная ветка разработки хранится в репозитории — локальном или удаленном, — к которому организован доступ всех участников. Когда разработчики вносят правки, VCS позволяет регистрировать эти изменения, объединять результаты работы разных специалистов, а при необходимости — выполнять возврат к любой из более ранних версий проекта.В классической модели контроля версий применяется централизованный подход: все файлы хранятся в едином репозитории, а управление версиями обеспечивается выделенным сервером. Участник проекта перед началом работы с помощью специальных команд запрашивает актуальную или нужную ему версию файлов. Завершив внесение правок, он отправляет обновлённую версию обратно в хранилище. При этом все предыдущие версии сохраняются в центральном репозитории, и к ним можно обратиться в любой момент. Чтобы экономить дисковое пространство, сервер может не сохранять полностью каждый изменённый файл, а применять дельта-компрессию — записывать только различия между последовательными версиями.Gitflow Workflow, семантическое версионирование и Conventional CommitsGitflow Workflow — это модель ветвления для Git, которая была опубликована и популяризована Винсентом Дриссеном. Данная модель предполагает выстраивание строгой структуры веток с учётом выпуска версий проекта. Gitflow Workflow отлично подходит для организации рабочего процесса, основанного на регулярных релизах. Работа по этой модели вклю-

чает создание отдельных веток для разработки новых функций, подготовки релизов, а также для срочного исправления ошибок в рабочей среде, что позволяет поддерживать стабильность основной ветки проекта на всех этапах разработки.Семантическое версионирование, или SemVer, описывается в одноимённом манифесте и представляет собой систему правил для присвоения номеров версий. Кратко его можно описать следующим образом: версия задаётся в виде кортежа МАЖОРНАЯ_ВЕРСИЯ.МИНОРНАЯ_ВЕРСИЯ.ПАТЧ. Номер версии следует увеличивать по определённым правилам. МАЖОРНУЮ версию увеличивают в том случае, когда в проект вносятся обратно несовместимые изменения API. МИНОРНУЮ версию увеличивают при добавлении новой функциональности, которая не нарушает обратной совместимости. ПАТЧ-версию увеличивают, когда выполняются обратно совместимые исправления ошибок. Дополнительные обозначения для предрелизных версий и билд-метаданных могут быть добавлены как расширение к основному формату МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ, например, в виде суффиксов -alpha, -beta или +build.15.Спецификация Conventional Commits представляет собой соглашение о том, как нужно писать сообщения коммитов. Данная спецификация полностью совместима с семантическим версионированием и даже более того — она тесно с ним связана. Conventional Commits регламентирует структуру сообщения коммита и определяет основные типы изменений, такие как feat для новой функциональности, fix для исправления ошибок, BREAKING CHANGE для изменений, нарушающих обратную совместимость, а также вспомогательные типы вроде docs, style, refactor и другие. Соблюдение этой специnebсфикации позволяет автоматически определять следующую версию проекта в соответствии с правилами SemVer, а также генерировать changelog на основе истории коммитов, что значительно упрощает управление версиями и выпуск релизов.

4 Выполнение лабораторной работы

4.1 Установка репозиториев.

Установка из колекции репозиториев **Corp** (рис. 4.1). (рис. 4.2).

```
sasha@adsemyonov:~$ sudo dnf copr enable elegos/gitflow
```

Рисунок 4.1: Установка из коллекции репозиториев

```
sasha@adsemyonov:~$ wget -q https://raw.githubusercontent.com/petervanderdoes/gitflow-avh/dev
elop/contrib/gitflow-installer.sh
sasha@adsemyonov:~$ ls
 gitflow-installer.sh            work    Документы   Изображения   Общедоступные   Шаблоны
 quarto-1.6.40-linux-amd64.deb   Видео   Загрузки    Музыка        'Рабочий стол'
sasha@adsemyonov:~$ sudo bash gitflow-installer.sh
[sudo] пароль для sasha:
### git-flow no-make installer ###
Usage: [environment] gitflow-installer.sh [install|uninstall] [stable|develop|version] [tag]
Environment:
   PREFIX=/usr/local
   REPO_HOME=https://github.com/petervanderdoes/gitflow-avh.git
   REPO_NAME=gitflow
sasha@adsemyonov:~$ sudo bash gitflow-installer.sh stable
### git-flow no-make installer ###
Usage: [environment] gitflow-installer.sh [install|uninstall] [stable|develop|version] [tag]
Environment:
   PREFIX=/usr/local
   REPO_HOME=https://github.com/petervanderdoes/gitflow-avh.git
   REPO_NAME=gitflow
```

Рисунок 4.2: Установка из коллекции репозиториев

Установка **node.js** (рис. 4.3).

```
sasha@adsemyonov:~$ sudo dnf install nodejs
Обновление и загрузка репозиториев:
 Fedora 42 - x86_64                              100% |  21.1 KiB/s |  23.1 KiB |  00m01s
 Fedora 42 - x86_64 - Updates                    100% |  11.3 KiB/s |   5.8 KiB |  00m01s
 Copr repo for gitflow owned by elegos           100% | 806.0   B/s |   2.4 KiB |  00m03s
Репозитории загружены.
Пакет                        Арх.     Версия                      Репозиторий         Размер
Установка:
```

Рисунок 4.3: Установка node.js

Установка **pnpm** (рис. 4.4).

```
sasha@adsemyonov:~$ sudo dnf install pnpm
Обновление и загрузка репозиториев:
Репозитории загружены.
Пакет                    Арх.     Версия                    Репозиторий         Размер
Установка:
 pnpm                    noarch   10.9.0-1.fc42             updates           16.3 MiB

Сводка транзакции:
 Установка:         1 пакета

Общий размер входящих пакетов составляет 3 MiB. Необходимо загрузить 3 MiB.
После этой операции будут использоваться дополнительные 16 MiB (установка 16 MiB, удаление 0
B).
Is this ok [y/N]: y
[1/1] pnpm-0:10.9.0-1.fc42.noarch                  100% |   2.3 MiB/s |   3.1 MiB |  00m01s
--------------------------------------------------------------------------------------------
[1/1] Всего                                     100% |   1.5 MiB/s |   3.1 MiB |  00m02s
```

Рисунок 4.4: Установка pnpm

Выполним **source~/.bashrc** (рис. 4.5).

```
sasha@adsemyonov:~$ source ~/.bashrc
```

Рисунок 4.5: Выполнение команды

4.2 Общепринятые коммиты.

Данную комнаду мы выполняем дл помощи в форматировании коммитов (рис. 4.6).

```
sasha@adsemyonov:~$ pnpm add -g commitizen
```

Рисунок 4.6: Выполнение pnpm add -g commitizen

Выполним команды для помощи в создании логов (рис. 4.7).

```
sasha@adsemyonov:~$ pnpm add -g standard-changelog
 WARN  2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Packages: +30
++++++++++++++++++++++++++++++
```

Рисунок 4.7: Выполнение команды pnpm add -g standard-changelog

4.3 Создание репозитория git.

Создадим репозиторий на **GitHub** и назовем его **git-extended** (рис. 4.8).

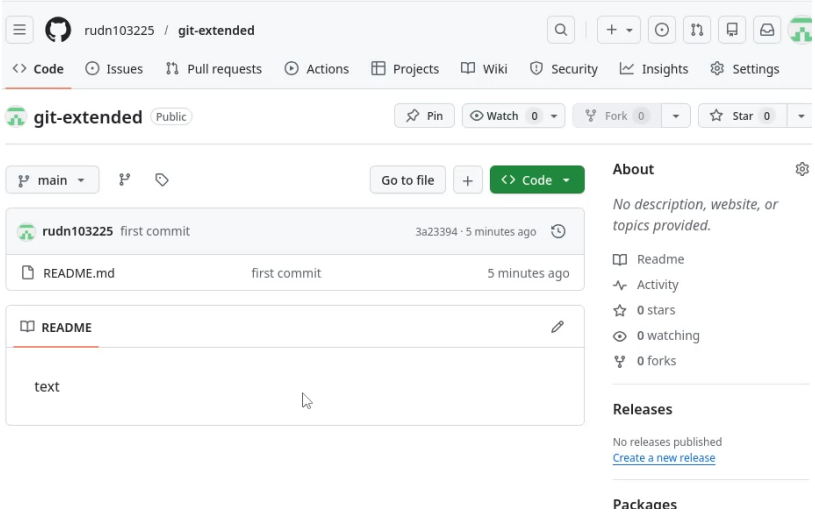
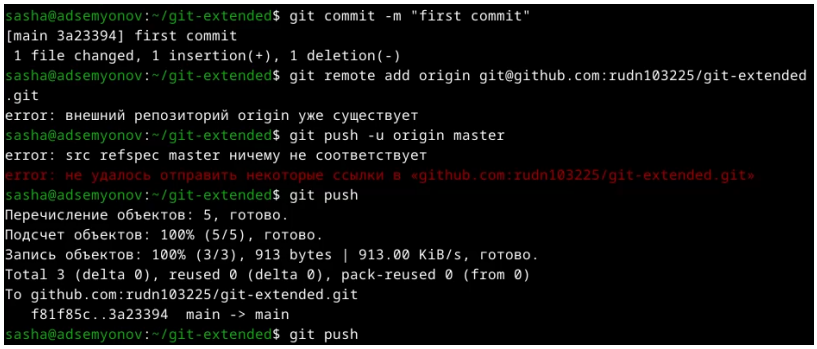


Рисунок 4.8: Созданный репозиторий

Делаем первый коммит и выкладываем на **GitHub** (рис. 4.9).



```
sasha@adsemyonov:~/git-extended$ git commit -m "first commit"
[main 3a23394] first commit
 1 file changed, 1 insertion(+), 1 deletion(-)
sasha@adsemyonov:~/git-extended$ git remote add origin git@github.com:rudn103225/git-extended
.git
error: внешний репозиторий origin уже существует
sasha@adsemyonov:~/git-extended$ git push -u origin master
error: src refspec master ничему не соответствует
error: не удалось отправить некоторые ссылки в «github.com:rudn103225/git-extended.git»
sasha@adsemyonov:~/git-extended$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
Запись объектов: 100% (3/3), 913 bytes | 913.00 KiB/s, готово.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:rudn103225/git-extended.git
   f81f85c..3a23394  main -> main
sasha@adsemyonov:~/git-extended$ git push
```

Рисунок 4.9: Выполнение git commit -m “first commit и git push -u origin master

Конфигурация общепринятых коммитов (рис. 4.10).

```
{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "Git repo for educational purposes",
  "main": "index.js",
  "repository": "git@github.com:rudn103225/git-extended.git",
  "author": "Alexander Semyonov <1032252587@pfur.ru>",
  "license": "CC-BY-4.0",
  "config": {
        "commitizen": {
                "path": "cz-conventional-changelog"
        }
    }
}
```

Рисунок 4.10: Выполнение pnpm init и заполнение пакетов

Добавим новые файлы (рис. 4.11).

```
sasha@adsemyonov:~/git-extended$ git add .
```

Рисунок 4.11: Созданный репозиторий

Выполним коммит (рис. 4.12).

```
sasha@adsemyonov:~/git-extended$ git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0
```

Рисунок 4.12: Коммит

Отправим на **GitHub** (рис. 4.13).

```
sasha@adsemyonov:~/git-extended$ git push
Перечисление объектов: 4, готово.
Подсчет объектов: 100% (4/4), готово.
При сжатии изменений используется до 3 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (3/3), 1.17 KiB | 1.17 MiB/s, готово.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:rudn103225/git-extended.git
   3a23394..da6786e  main -> main
```

Рисунок 4.13: Отправка ни гит

4.4 Конфигурация git flow.

Инициализируем **git flow** (рис. 4.14).

```
sasha@adsemyonov:~/git-extended$ git flow init

Which branch should be used for bringing forth production releases?
   - main
Branch name for production releases: [main]
```

Рисунок 4.14: Инициализация

Проверяем, что мы на ветке **develop** (рис. 4.15).

```
sasha@adsemyonov:~/git-extended$ git branch
* develop
  main
```

Рисунок 4.15: Проверка ветки

Загрузка всего репозитория в хранилище (рис. 4.16).

```
sasha@adsemyonov:~/git-extended$ git push --all
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/rudn103225/git-extended/pull/new/develop
remote:
To github.com:rudn103225/git-extended.git
 * [new branch]      develop -> develop
```

Рисунок 4.16: Загрузка

Установим внешнюю ветку как вышестоящую для этой ветки (рис. 4.17).

```
sasha@adsemyonov:~/git-extended$ git flow release start 1.0.0
Переключились на новую ветку «release/1.0.0»

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparin
- When done, run:

     git flow release finish '1.0.0'
```

Ножницы

Снимок экрана скопирован в буфер
Автоматически сохраняются в папк

Рисунок 4.17: установка вышестоящей ветки

Созданем релиз с версией **1.0.0** (рис. 4.18).

```
sasha@adsemyonov:~/git-extended$ git flow release start 1.0.0
Переключились на новую ветку «release/1.0.0»

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparir
- When done, run:

     git flow release finish '1.0.0'
```

Ножницы

Снимок экрана скопирован в буфер
Автоматически сохраняются в папк

Рисунок 4.18: Создание релиза

Создадим журнал изменений (рис. 4.19).

```
sasha@adsemyonov:~/git-extended$ standard-changelog --first-release
```

Рисунок 4.19: Создание журнала

Добавим журнал изменений в индекс (рис. 4.20).

```
sasha@adsemyonov:~/git-extended$ git add CHANGELOG.md
sasha@adsemyonov:~/git-extended$ git commit -am 'chore(site): add changelog'
[release/1.0.0 0650bb6] chore(site): add changelog
 1 file changed, 1 insertion(+)
 create mode 100644 CHANGELOG.md
```

Рисунок 4.20: Добавление журнала в индекс

Зальём релизную ветку в основную ветку (рис. 4.21).

```
sasha@adsemyonov:~/git-extended$ git flow release finish 1.0.0

Переключились на ветку «main»
Эта ветка соответствует «origin/main».
Merge made by the 'ort' strategy.
 CHANGELOG.md | 1 +
```

Рисунок 4.21: Залив релизной ветки

Отправим данные на **GitHub** (**?@fig-22**). (**?@fig-23**).

```
sasha@adsemyonov:~/git-extended$ git push --all
Перечисление объектов: 6, готово.
Подсчет объектов: 100% (6/6), готово.
При сжатии изменений используется до 3 потоков
Сжатие объектов: 100% (4/4), готово.
Запись объектов: 100% (5/5), 2.66 KiB | 2.66 MiB/s, готово.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:rudn103225/git-extended.git
```

Рисунок 4.22: Отправка данных

```
sasha@adsemyonov:~/git-extended$ git push --tags
Перечисление объектов: 1, готово.
Подсчет объектов: 100% (1/1), готово.
Запись объектов: 100% (1/1), 164 bytes | 164.00 KiB/s, готово.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:rudn103225/git-extended.git
```

Рисунок 4.23: Отправка данных

Создадим релиз на **GitHub**, а для этого будем использовать утилиты с **GitHub**. (рис. 4.24).

```
sasha@adsemyonov:~/git-extended$ gh release create v1.0.0 -F CHANGELOG.md
https://github.com/rudn103225/git-extended/releases/tag/v1.0.0
```

Рисунок 4.24: Создание релиза с помощью утилитов

4.5 Работа с репозиторием git.

Создадим ветку для новой функциональности (рис. 4.25).

```
sasha@adsemyonov:~/git-extended$ git flow feature start feature_branch
Переключились на новую ветку «feature/feature_branch»

Summary of actions:
- A new branch 'feature/feature_branch' was created, based on 'develop'
- You are now on branch 'feature/feature_branch'

Now, start committing on your feature. When done, use:

     git flow feature finish feature_branch
```

Рисунок 4.25: Создание новой ветки

Затем продолжаеем работу как обычно (рис. 4.26).

```
sasha@adsemyonov:~/git-extended$ git branch
  develop
* feature/feature_branch
  main
```

Рисунок 4.26: Продолжение работы

По окончанию разработки новой функциональности слудющим шагом следует объеденить ветку **feature_branch** с **develop** (рис. 4.27).

```
sasha@adsemyonov:~/git-extended$ git flow feature finish feature_branch

Переключились на ветку «develop»
Эта ветка соответствует «origin/develop».
Уже актуально.
Ветка feature/feature_branch удалена (была 9479424).

Summary of actions:
- The feature branch 'feature/feature_branch' was merged into 'develop'
- Feature branch 'feature/feature_branch' has been locally deleted
```

Рисунок 4.27: Объединение веток

Создадим релиз, но уже с версией **1.2.3** (рис. 4.28).

```
sasha@adsemyonov:~/git-extended$ git flow release start 1.2.3
Переключились на новую ветку «release/1.2.3»

Summary of actions:
- A new branch 'release/1.2.3' was created, based on 'develop'
- You are now on branch 'release/1.2.3'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:
```

Рисунок 4.28: Создание релиза

Нужно обновить номер **version** на **1.2.3** в файле **package.json** (рис. 4.29).

```
{
  "name": "git-extended",
  "version": "1.2.3",
  "description": "Git repo for educational purposes",
  "main": "index.js",
  "repository": "git@github.com:rudn103225/git-extended.git",
  "author": "Alexander Semyonov <1032252587@pfur.ru>",
  "license": "CC-BY-4.0",
  "config": {
        "commitizen": {
                "path": "cz-conventional-changelog"
        }
    }
}
```

Рисунок 4.29: Обновление номера версии

Создаем журнал изменений (рис. 4.30).

```
sasha@adsemyonov:~/git-extended$ standard-changelog
```

Рисунок 4.30: Создание журнала

Добавим журнал изменений (рис. 4.31).

```
sasha@adsemyonov:~/git-extended$ git add CHANGELOG.md
sasha@adsemyonov:~/git-extended$ git commit -am 'chore(site): update changelog'
[release/1.2.3 398e767] chore(site): update changelog
 2 files changed, 3 insertions(+), 1 deletion(-)
```

Рисунок 4.31: Добавление журнала

Зальём релизную ветку в основную ветку (рис. 4.32).

```
sasha@adsemyonov:~/git-extended$ git flow release finish 1.2.3
Переключились на ветку «main»
Эта ветка соответствует «origin/main».
Merge made by the 'ort' strategy.
 CHANGELOG.md | 2 ++
 package.json | 2 +-
 2 files changed, 3 insertions(+), 1 deletion(-)
Уже на «main»
Ваша ветка опережает «origin/main» на 3 коммита.
  (используйте «git push», чтобы опубликовать ваши локальные коммит
Переключились на ветку «develop»
```

Рисунок 4.32: Залив ветки

Отправим данные на **GitGub** (рис. 4.33). (рис. 4.34).

```
sasha@adsemyonov:~/git-extended$ git push --all
Перечисление объектов: 9, готово.
Подсчет объектов: 100% (9/9), готово.
При сжатии изменений используется до 3 потоков
Сжатие объектов: 100% (6/6), готово.
Запись объектов: 100% (6/6), 2.79 KiB | 571.00 KiB/s, готово.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local ob
```

Рисунок 4.33: Отправка

```
sasha@adsemyonov:~/git-extended$ git push --tags
Перечисление объектов: 1, готово.
Подсчет объектов: 100% (1/1), готово.
Запись объектов: 100% (1/1), 164 bytes | 164.00 KiB/s, готово.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:rudn103225/git-extended.git
```

Рисунок 4.34: Отправка

Создадим релиз на **GitHub** с комментарием из журнала изменений (рис. 4.35).

```
sasha@adsemyonov:~/git-extended$ gh release create v1.2.3 -F CHANGELOG.md
https://github.com/rudn103225/git-extended/releases/tag/v1.2.3
```

Рисунок 4.35: Создание релиза

5 Выводы

В ходе выполнения лабораторной работы я получила навыки работы с репозиториями **git**.

Список литературы

ТУИС